

Cascading Referential Integrity Constraints in SQL Server

In this article, I am going to discuss the **Cascading Referential Integrity Constraints in SQL Server** with examples. Please read our previous article where we discussed [How to make Primary Key and Foreign key relationships between more than two tables in SQL Server](#) with examples. As part of this article, we are going to discuss the following pointers in detail.

1. [What are Cascading Referential Integrity Constraints in SQL Server?](#)
2. [What are the Actions Performed By SQL Server?](#)
3. [Example of Cascading Referential Integrity Constraints in SQL Server.](#)
4. [Understanding Self Referential Integrity Constraint in SQL Server?](#)

What are Cascading Referential Integrity Constraints in SQL Server?

The Cascading Referential Integrity Constraints in SQL Server are the foreign key constraints that tell SQL Server to perform certain actions whenever a user attempts to delete or update a primary key to which an existing foreign keys point.

What are the Actions Performed By SQL Server?

In order to tell the SQL Server what actions to perform whenever a user trying to delete or update a primary key value to which existing foreign key points, we are provided with the following options while working with Cascading Referential Integrity Constraints

SET NULL:

If a user tries to delete or update statement(s) that will affect rows in the foreign key table, then those values will be set to NULL when the primary key record is deleted or updated in the Primary key table. The important thing that we need to keep in mind that the foreign key columns affected must allow NULL values.

CASCADE:

If a user tries to delete the statement(s) which will affect the rows in the foreign key table, then those rows will be deleted when the primary key record is deleted. Similarly, if an update statement affects rows in the foreign key table, then those rows will be updated with the value from the primary key record after it has been updated.

SET DEFAULT:

If a delete or update statement affects rows in a foreign key table, then all rows containing those foreign keys are set to the default value. All foreign key columns in the related table must have default constraints defined on them.

NO ACTION:

This is the default action that SQL Server performs. This specifies that if an update or deletes statement affects rows in foreign key tables, then the action will be denied and rolled back. An error message will be raised.

Example of Cascading Referential Integrity Constraints in SQL Server:

Let us understand the Cascading Referential Integrity Constraints in SQL Server with one example. In order to understand this, we need two database tables. So let's first create two tables (Person and Gender) and enforce primary and foreign key constraints as shown below.

Create Gender Table

```
CREATE TABLE Gender
(
    Id      INT PRIMARY KEY,
    Gender  NVARCHAR(50)
)
-- Insert some test data in Gender Table
Insert into Gender values (1, 'Male')
Insert into Gender values (2, 'Female')
Insert into Gender values (3, 'Unknown')
```

Create Person Table

```
CREATE TABLE [Person](
    [Id] [int] PRIMARY KEY,
    [Name] [varchar](100) NOT NULL,
    [Email] [varchar](100) NOT NULL,
    [GenderID] [int] NULL
)
```

In the **Person** table, the **GenderID** column is the **foreign key** which is referencing the **ID** column of the **Gender** table. The Foreign key constraints can be added graphically using SSMS or using a query as shown below.

```
-- Add a foreign key reference using query
Alter table Person
add constraint Person_GenderId_FK FOREIGN KEY (GenderId) references Gender(ID)
--Insert some test data in Person Table
Insert into Person values (1, 'abc','abc@gmail.com',1)
Insert into Person values (2, 'pqr','pqr@gmail.com',2)
Insert into Person values (3, 'xyz','xyz@gmail.com',3)
```

For example, consider the above 2 tables (Person and Gender). If we delete the row with **ID = 1** from the **Gender** table then the row with **ID = 1** from the **Person** table becomes an **orphan record**. We will not be able to tell the Gender for this row. So, the Cascading referential integrity constraint in SQL Server can be used to define the actions that Microsoft SQL Server should take. By default, **we get an error and the DELETE or UPDATE statement is rolled back**.

If we want to delete or update a record in the parent table (in this case **Gender** table) when they have a corresponding child record in the child table (in this case **Person** table) we are provided with a set of rules to perform delete and update operations known as **DELETE** rules and **UPDATE** rules as shown in the below image.

DELETE Rules:

1. ON DELETE NO ACTION
2. ON DELETE CASCADE
3. ON DELETE SET NULL
4. ON DELETE SET DEFAULT

UPDATE Rules:

1. ON UPDATE NO ACTION
2. ON UPDATE CASCADE
3. ON UPDATE SET NULL
4. ON UPDATE SET DEFAULT

Note: The Delete rules and update rules were not imposed on the master table, they are imposed on the child table that too on the foreign key column. Please have a look at the following diagram for the syntax.

```
<COLUMN> <DATATYPE> <CONSTRAINT NAME> REFERENCES <MASTER TABLE  
NAME> (<PRIMARY KEY COLUMN>)  
ON DELETE {NO ACTION / CASCADE / SET NULL / DEFAULT}  
ON UPDATE {NO ACTION / CASCADE / SET NULL / DEFAULT}
```

Example:

Let's delete the existing Person table and again create the person table as shown below

```
CREATE TABLE [Person](  
    [Id] [int] PRIMARY KEY,  
    [Name] [varchar](100) NOT NULL,  
    [Email] [varchar](100) NOT NULL,  
    [GenderID] [int] CONSTRAINT FK_Person_GenderID FOREIGN KEY REFERENCES dbo.Ge  
    ON DELETE CASCADE  
    ON UPDATE CASCADE  
)
```

Now insert the following data into the Person table

```
--Insert some test data in Person Table  
Insert into Person values (1, 'abc', 'abc@gmail.com', 1)  
Insert into Person values (2, 'pqr', 'pqr@gmail.com', 2)  
Insert into Person values (3, 'xyz', 'xyz@gmail.com', 3)
```

After making the relationship with cascade rules now we can perform the update and delete operations on parent table reference column data and affected the corresponding child table reference column data also.

Examples:

Insert into Person values (4, 'pranaya', 'pranaya@g.com', 4) — Not Allowed

When we execute the above query it will give us the below error

The INSERT statement conflicted with the FOREIGN KEY constraint "FK_Person_GenderID". The conflict occurred in database "TestDB", table "dbo.Gender", column 'Id'.

The statement has been terminated.

DELETE FROM Gender WHERE Id = 2 — Allowed

Once we delete the record from the Gender table with Id = 2, it also automatically delete the record(s) from the Person table where GenderId = 2. Fetch the records from the Person table as **Select * from Person**

Id	Name	Email	Gender...
1	abc	abc@gmail.com	1
3	xyz	xyz@gmail.com	3

UPDATE Gender SET Id = 4 where Gender = 'Male' — Allowed

Once we update the record from the Gender table with Id = 2 where Gender = 'Male', it also automatically updates the record(s) in the Person table. Fetch the records from the Person table and observe.

Id	Name	Email	GenderID
1	abc	abc@gmail.com	4
3	xyz	xyz@gmail.com	3

What is Self Referential Integrity Constraint in SQL Server?

This is the same as the referential integrity we have learned earlier. In earlier cases, we are binding one column of a table with another column of another table whereas in self-referential integrity we bind a column of a table with another column of the same table i.e. both the foreign key and primary key will be present in one table only.

Let us see an example for understanding this concept.

We have the following employee table having the following records.

Primary key column EmpID	Ename	Job	Foreign key column ManagerID
1000	AAA	PRESIDENT	NULL
1001	BBB	MANAGER	1000
1002	CCC	SALESMAN	1001
1003	DDD	CLERK	1001
1004	EEE	MANAGER	1000
1005	FFF	ANALYST	1004
1006	GGG	CLERK	1010 (INVALID)

In the above table, we are binding the column ManagerID (foreign) with another column of the same table i.e. EmpID (Reference key) to verify the value entered into the ManagerID column to be existing under EmpNo column

Creating the Employee table using all constraints in column level

```
CREATE TABLE Employee
(
    EmpID INT CONSTRAINT EmpID_PK PRIMARY KEY,
    Ename VARCHAR(50) NOT NULL,
    Job VARCHAR(50),
    ManagerID INT CONSTRAINT ManagerID_FK REFERENCES Employee(EmpID),
    Salary MONEY DEFAULT 3000 CONSTRAINT Salary_CK CHECK (Salary BETWEEN 5000
)
```

In the next article, I am going to discuss the [Identity column in SQL Server](#) with examples. Here, in this article, I try to explain **Cascading Referential Integrity Constraints in SQL Server** with Examples. I hope you enjoy this Cascading Referential Integrity Constraints in SQL Server with Examples article.